

lab4 — E-Commerce 쿼리 튜닝 리포트

DB skala_db4 / 스키마 ecom · PostgreSQL 17. 각 문항을 「작성 → EXPLAIN ANALYZE 병목 파악 → 인덱스/조인/MV 처방 → 재 측정」순으로 튜닝.

측정 기준: EXPLAIN (ANALYZE, BUFFERS) 2회차 값, 비교는 Execution Time. 시드가 random() 기반이라 수치는 스냅샷. 근 거 캡처: captures/Q0X_{before,after,joins}.txt.

1. 문항별 요약

- Q01 · 최근 1개월 실판매 총액** — 개선 8.36 → 4.99ms (~40%↓)
 - 병목: COUNT(DISTINCT)가 부른 Sort(219kB)
 - 처방: 불필요한 컬럼 제거 → Sort 소멸
- Q02 · 월별 주문수 / 매출 / AOV** — 개선 12.16 → 2.07ms (~83%↓)
 - 병목: COUNT(DISTINCT) Sort(952kB) + order_items 18,700행 조인 스캔
 - 처방 ①: 매출은 MV 월 롤업 · 건수는 orders 단독 집계로 분리(조인·Sort 제거)
 - 처방 ②: 부분 커버링 인덱스로 Index-Only Scan
- Q03 · 최근 90일 카테고리 Top10** — 개선 12.35 → 1.11ms (~91%↓)
 - 병목: order_items 18,700행 조인 + 집계(Hash Join). order_ts 61% 통과라 인덱스 기각
 - 처방: 카테고리×일 사전집계 MV(mv_category_daily)로 조인 대체
- Q04 · 제품별 누적매출 RANK Top20** — 개선 10.76 → 0.55ms (~95%↓)
 - 병목: order_items 18,700행 조인 → 제품 집계(Hash Join). 누적이라 날짜필터 없음 → 인덱스 무의미
 - 처방: 제품 누적매출 MV(mv_product_revenue, 600행)로 조인 대체 후 RANK
- Q05 · 고객 RFM** — 개선 17.85 → 5.54ms (~69%↓)
 - 병목: order_items 조인 + COUNT(DISTINCT) Sort(1065kB) + NTILE 3중 정렬
 - 처방: 고객 집계 MV(mv_customer_rev)로 조인·대형 Sort 제거, NTILE 등급은 조회 시 부여
- Q06 · 첫 구매 후 30일 내 재구매율** — 개선 8.55 → 5.98ms (~30%↓), Buffers 142 → 74
 - 병목: orders 2회 Seq Scan + self-join(30일 window가 Join Filter)
 - 처방: 쿼리 재작성 — MIN() OVER로 첫주문일 부여 → 단일 스캔, self-join 제거(MV 미사용)
- Q07 · 품질 위험 재고** — 0.385ms (튜닝 불필요)
 - 병목: 없음 — inventory(600)×products(600), 이미 ~0.4ms
 - 처방: 변경 없음 — 부분 인덱스 검토 후 기각(소규모라 Seq Scan 최적, 강제 시 더 느림)
- Q08 · 효자상품(평점 4.5↑ · 리뷰 50↑)** — 개선 1.94 → 0.79ms (~59%↓)
 - 병목: reviews(2,031)×products 조인을 먼저 → 2,031행에 상품정보 실어 GROUP BY
 - 처방: 집계-후-조인 재작성 — reviews 먼저 집계 + HAVING(12개) → 살아남은 12개만 조인
- Q09 · 쿠폰 사용/미사용 AOV 비교** — 개선 16.04 → 6.95ms (~57%↓)
 - 병목: order_items×orders(14,522행) 위 COUNT(DISTINCT) 용 Sort(1030kB, ~9ms)
 - 처방: 매출/건수 분리(Q02 패턴) — 매출은 조인 후 2그룹 SUM, 주문수는 orders 단독 COUNT → DISTINCT·Sort 소멸
 - 인사이트: 쿠폰 사용 주문 AOV(1,837)가 미사용(642)의 약 2.9배
- Q10 · 상위 1% 고객의 최근 60일 매출** — 개선 14.33 → 7.06ms (~51%↓)
 - 병목: customer_total 대전집계(order_items 18,700×orders → 2,246고객)로 상위 1% 판별
 - 처방: MV 재사용 — 랭킹을 mv_customer_rev.monetary로 대체, 최근 60일 매출만 실패데이터 조인
- Q11 · 안전 나눗셈으로 평균 안전 계산** — 개선 4.10 → 1.28ms (~69%↓)
 - 병목: 리뷰 0건 상품 479개 → 수동 평균에서 0/0 division by zero
 - 처방: 함수 선택 — sql_safe_div는 인라인(=NULLIF 동일 1.28ms), plpgsql f_safe_div는 미인라인(2.87ms) → safe_div 채택

2. 인덱스 추가 / 개선

스크립트: tuning/indexes.sql. 핵심 원칙 — 「인덱스는 쿼리를 커버해야 타고, 소규모·저선택도에선 Seq Scan이 최적」.

- Q02 · 적용** — idx_orders_rev_ts = orders(order_ts) WHERE status IN(3) (부분 커버링)
 - 월별 주문건수 집계가 Seq Scan → Index Only Scan(Heap Fetches 0). order_ts·상태만 필요해 인덱스가 쿼리를 완전 커버 → 힙·필터 생략. 4.69 → 2.07ms
 - 단, Index-Only 이득은 VACUUM/가시성맵 최신에 의존
- Q01 · 미적용** — 동일 인덱스 후보 검토
 - line_total(order_items 조인)이 필요해 인덱스가 커버 못 함 + 21% 선택도 → 플래너가 Seq Scan 유지. "같은 인덱스라도 쿼리가 커버돼야 탄다"는 대비 사례
- Q04 · 생성했으나 미사용** — idx_mv_product_revenue_rev = mv_product_revenue(revenue DESC)
 - RANK의 ORDER BY용이지만 MV가 600행이라 플래너가 Seq Scan+quicksort 선택. ORDER BY+인덱스 이득은 규모 의존 (행 수가 커지면 Sort 제거에 기여)
- Q07 · 검토 후 기각** — idx_inventory_at_risk = inventory(product_id) INCLUDE(qty, reorder) WHERE qty<reorder (부분 커버링)
 - inventory 600행이라 Seq Scan(5 buffers)이 인덱스+힙보다 저렴 → 플래너 거부, 강제 사용 시 cost 12.5 → 16으로 더 느림. "부분 인덱스는 대형·저선택도에서만 의미"
- Q08 · 검토 후 기각** — idx_reviews_prod_rating = reviews(product_id) INCLUDE(rating) (커버링)
 - reviews 2,031행 전건 집계라 Seq Scan(21 buffers)이 Index-Only Scan보다 저렴 → 플래너 미사용. Q08은 인덱스 대신 집계-후-조인 재작성으로 해결

3. Join 전략 비교 및 적용 (HJ / SMJ / NLJ)

Q01~Q04는 order_items ⋈ orders 등가 조인, Q06은 orders ⋈ orders self-join.
enable_hashjoin/enable_mergejoin/enable_nestloop 스위치로 3전략 강제 비교 → cost는 항상 HJ가 최소, 플래너가 HJ 선택.

문항별 cost (HJ=선택 / SMJ / NLJ)

- Q01 items⋈orders(최근 1개월): HJ **651** / SMJ 1001 / NLJ 1610 — NLJ Buffers 4363(=17배)
- Q02 items⋈orders(전체 매출): HJ **685** / SMJ 1149 / NLJ 3056
- Q03 items⋈orders⋈products⋈cat(90일): HJ **903** / SMJ 2172 / NLJ 4417 — NLJ 실측도 최악(14.8ms)
- Q04 items⋈orders(누적): HJ **795** / SMJ 1311 / NLJ 3363
- Q06 orders⋈orders self(30일 window): HJ **442** / SMJ 1019 / NLJ 1607 — NLJ Buffers 5919(=41배), window는 Join Filter
- Q07 inventory⋈products(600×600): HJ **29.6** / SMJ 59.9 / NLJ 115 — NLJ Buffers 185(=15배)
- Q08 reviews(2031)⋈products: HJ **67.2** / SMJ 219.5 / NLJ 130 — NLJ+Memoize Buffers 384(=13.7배)
- Q09 items⋈orders(매출): HJ **649** / SMJ 1113 / NLJ 3019 — NLJ Buffers 16176(=62배), 지배 비용은 조인 위 COUNT(DISTINCT) Sort
- Q10 items⋈orders(고객 집계): HJ **649** / SMJ 1113 / NLJ 3019 — NLJ Buffers 16168(=62배)
- Q11 products(600)⋈prod_rev(121): 소규모라 3방식 모두 sub-0.15ms 동일권(핵심은 조인 아닌 함수 선택)

해석

- **HJ가 정답**: 대형×대형 등가 조인 → 양쪽 1회 스캔 후 해시. cost 최소이자 규모 안정적
- **NLJ의 함정**: 작고 캐시된 데이터에선 wall-clock이 빠를 때가 있으나 Buffers가 최대 62배까지 폭발 → 데이터↑/캐시미스 시 위험. cost·buffers·loops로 판단
- **SMJ**: order_items가 idx_order_items_order로 이미 정렬돼 나쁘지 않지만, 반대쪽 정렬 비용 탓에 HJ보다 cost 높음

적용 결과

- 강제 전환한 쿼리는 없음 — 모든 문항에서 플래너의 HJ가 최적이라 유지
- 더 나아가 「조인 자체를 없애거나 줄이는」 것이 최종 튜닝. 수단은 문항 성격별로 갈림:
 - Q02~Q04: MV 사전집계로 order_items ⋈ orders 조인을 MV 읽기로 대체(무거운 팩트 조인)
 - Q06: MIN() OVER (PARTITION BY customer_id) 재작성으로 self-join을 단일 스캔으로 대체(자기참조)
 - Q08: 집계-후-조인 재작성으로 조인 입력을 2,031행 → 12행으로 축소

보충 — 함수·표현식 최적화(Q11): sql_safe_div는 플래너가 인라인 → 순수 NULLIF와 동일 1.28ms, plpgsql f_safe_div는 인라인 불가라 행당 호출로 2.87ms(~2.2배). '리뷰 없음'은 NULL이 의미상 맞아 safe_div 채택.

4. Materialized View

활용 요약 (MV → 활용 쿼리 → 효과)

- mv_daily_gmv → Q02: order_items 조인 대신 MV(124행) 월 롤업 → 12.16 → 4.69ms · 스크립트 배포 스키마 + tuning/mv_daily_gmv.sql
- mv_category_daily → Q03: order_items 18,700행 조인 대신 MV(~890행) → 12.35 → 1.11ms · tuning/mv_category_daily.sql
- mv_product_revenue → Q04: order_items 18,700행 조인 대신 MV(600행) → 10.76 → 0.55ms · tuning/mv_product_revenue.sql
- mv_customer_rev → Q05 + Q10: Q05 조인+COUNT(DISTINCT) 제거 17.85 → 5.54ms / Q10 랭킹 대전집계 대체 14.33 → 7.06ms · tuning/mv_customer_rev.sql

문항별 노트

- Q02: 매출은 MV, 건수는 orders 단독 집계로 분리(실시간 유지). orphan 주문 0건 확인 후 분리해도 결과 동일
- Q03: 카테고리×일 사전집계. MV는 day 단위라 '최근 90일'을 date_trunc('day', now()) 기준으로 맞춰야 원본과 일치(한낮 경계로 두면 경계일 누락)
- Q04: 제품 누적매출(전체 기간) 사전집계. RANK를 600행에만 적용
- Q05: 고객 R/F/M 원지표 사전집계. last_order_ts만 저장 → 경과일은 조회 시 계산(매일 최신), NTILE 등급도 조회 시 부여. 남은 비용은 NTILE 3중 정렬(전체 순위라 사전계산 불가)
- Q10: 같은 mv_customer_rev의 monetary로 상위 1% 랭킹만 대체. 최근 60일 매출은 실데이터 조인 → "누가 우량인가"는 MV 스냅샷(15:00), "최근 얼마 썼나"는 항상 최신. 하나의 MV가 두 문항(Q05·Q10)을 함께 가속하는 재사용 사례
- Q06: 일부터 MV를 만들지 않음 — 이미 <10ms 리포트라 self-join 제거(window 재작성)만으로 충분. "MV가 항상 답은 아니다"의 판단 사례

mv_daily_gmv 상세

- 정의: 매출상태 3종만 일별 sum(line_total) 사전집계 → 일별 총 판매금액 리포트 가속. 현재 124행(2026-03-26~08-01)
- 가속 실증: 원본 조인 집계 ~12~14ms(Buffers 252) → MV 조회 0.02ms(Buffers 1), 약 600배
- 실행 시점(조회·갱신·원본변경 분리): 조회 = 저장 스냅샷 읽기(재계산 안 함) / REFRESH만 재계산(오후 3시 스케줄러 하루 1회) / 원본 변경은 수시(REFRESH 전까지 stale)

갱신 전략

- 주기: 매일 15:00 1회 — 과거일은 확정값(불변), 당일치만 갱신 대상

- 신선도: 당일 데이터 최대 ~24h 지연 — 오후 리포팅 직전 1회 갱신으로 일괄 스냅샷
- 방식: REFRESH ... CONCURRENTLY — 갱신 중에도 조회 무차단(전체 REFRESH는 조회 잠금). 전제로 UNIQUE 인덱스 필요 (ux_mv_daily_gmv_day 추가 완료)
- 스케줄: pg_cron 미설치 → OS 스케줄러(launchd 권장 / cron 대안). 설계·문서만, 시스템 미설치. 대상 스크립트 tuning/refresh_mv_daily_gmv.sh(4개 MV 일괄 CONCURRENTLY 갱신)

5. 첨부 사진 파일명 (숫자 순서)

1. 1-1.png
2. 1-2.png
3. 2-1.png
4. 2-2.png
5. 3-1.png
6. 3-2.png
7. 4-1-1.png
8. 4-1-2.png
9. 4-2-1.png
10. 4-2-2.png
11. 5-1.png
12. 5-2.png
13. 6-1.png
14. 6-2.png
15. 7-1.png
16. 8-1.png
17. 8-2.png
18. 9-1.png
19. 9-2.png
20. 10-1-1.png
21. 10-1-2.png
22. 10-2.png
23. 11.png
24. 11-2.png